Chapter 8. Scripting, Part 1

Note: this part of the course is completely independent from the RedHat course material.

Pointers to more training material

- LinuxFun from chapter 23 onwards
- The Linux Command Line from chapter 24 onwards
- LPIC Linux Essentials 3.3
- LPIC 102
- https://quantchem.kuleuven.be/unix_manual/shell.html
- https://github.com/hpcleuven/Linux-scripting
- https://labex.io/tutorials/linux-linux-sh-command-with-practical-examples-422915
- ...

Recap

Unix Commands

- Every command is a program
- Every command has 2 inputs:
  - Standard input (`System.in`)
  - Command-line parameters (and options) ("arguments") (`args`)
- Every command has 3 outputs:
  - Standard output (`System.out`)
  - Standard error (`System.err`)
  - Exit-code (`System.exit()`)

Example in Java

Errors are written to stderr via `System.err` . The exit code is returned with the `System.exit` call.

```
public static void main(String[] args) {
  if (args.length != 0) {
    System.err.println("Error! Do not provide any parameters\n");
    System.exit(1);
  }
  String s;
  s = new Scanner(System.in).nextLine();
  System.out.printf("You entered the string '%s'\n", s);
}
```

Example in C

Errors are written to stderr. The exit code is returned as return value from main or with the `exit` call.

```c
#include <stdio.h>
int main(int argc, char** argv) {
  if (argc != 1) {
    fprintf(stderr, "Error! Do not provide any parameters\n");
    return 1;
  }
  char s[100];
  scanf("%s", s);
  printf("You entered the string '%s' in\n", s);
  return 0;
}
```

Basic Scripts

In the simplest terms, a shell script is a file containing a series of commands. The shell reads this file and carries out the commands as though they have been entered directly on the command line.
The shell is somewhat unique, in that it is both a powerful command line interface to the system and a scripting language interpreter. As we will see, most of the things that can be done on the command line can be done in scripts, and most of the things that can be done in scripts can be done on the command line.
We have covered many shell features, but we have focused on those features most often used directly on the command line. The shell also provides a set of features usually (but not always) used when writing programs.

Summarized:

- A script is a text file.
- A script can be written in different languages (bash, python, perl, ...), we write in bash.
- The simplest script = a list of commands.
- Handy for bundling commands and executing them regularly (automation).

Example: backup

- Scenario: You want to take regular backups.
- Commands:
  - `tar zcf /tmp/backup.tgz /root 2>/dev/null`
  - `sync` [1]
- Problems:
  - A lot of typing.

- Risk of typos.

Solution: script

- Create a text file `~/bin/backup1` with:

```
echo "Backing up..."
tar zcf /tmp/backup.tgz /root 2>/dev/null
sync
echo "Done."
```

- Make it "executable" with:

```
chmod +x ~/bin/backup1
```

- And execute it:
  `$ ~/bin/backup1`
  or
  `$ ./bin/backup1`
  We'll gradually expand this script in this part of the course.

Where to put scripts?

The `~/bin` directory is a good place to put scripts intended for personal use. If we write a script that everyone on a system is allowed to use, the traditional location is `/usr/local/bin`. Scripts intended for use by the system administrator are often located in `/usr/local/sbin`. In most cases, locally supplied software, whether scripts or compiled programs, should be placed in the `/usr/local` hierarchy and not in `/bin` or `/usr/bin`. These directories are specified by the Linux Filesystem Hierarchy Standard to contain only files supplied and maintained by the Linux distributor.

Which shell or interpreter?

When executing a script (bash or other), multiple shells (see `/etc/shells`) and other command line interpreters such as python or perl are available. How does Linux know which shell or interpreter to use?
Remember also that Linux does not use file extensions (.sh, .py, .pl..) to determine file type. And neither does Linux use file extension to determine which shell or interpreter to use.

Solution: Shebang

```
#!/bin/bash
echo "Backing up..."
tar zcf /tmp/backup.tgz /root 2>/dev/null
sync
echo "Done."
```

Other possibilities:

- Bourne shell: `#!/bin/sh`
- Perl: `#!/usr/bin/perl`
- Python: `#!/usr/bin/python3`

  ...

On the first line!

Make sure the shebang is on the first line:

❌ Wrong:

```
# This is a comment
#!/bin/bash
echo "Hello world"
```

✅ Right:

```
#!/bin/bash
# This is a comment
echo "Hello world"
```

Alternative

Another shebang alternative that is used:

```
#!/usr/bin/env bash
```

Bash is *not always* in `/bin` directory. Particularly on *non-Linux* systems or due to installation as an optional package.
With the above notation, the Operating System searches for the bash executable in directories, listed in the PATH environmental variable.

Other shells?

Here's a brief table comparing some popular Unix/Linux shells:

Shell Name	Full Name / Origin	Key Characteristics	Primary Use Case	Common as Default on (Examples)
sh	Bourne Shell	Original Unix shell; basic, foundational.	System scripting (basic), historical.	Unix systems (historically); often a symlink to another shell (`bash` , `dash`) on modern Linux.

Shell Name	Full Name / Origin	Key Characteristics	Primary Use Case	Common as Default on (Examples)
csh	C Shell	C-like syntax for scripting; interactive features.	Less common for general scripting; niche use.	FreeBSD (root shell by default).
ksh	Korn Shell	Superset of Bourne; blends Csh's interactive features with sh's scripting power; performant.	Enterprise environments, robust scripting.	AIX, HP-UX, Solaris (historically significant).
bash	Bourne-Again SHell	Most common modern shell; superset of sh; extensive features, good for interactive and scripting.	General purpose user shell, widespread scripting.	Most Linux distributions (e.g., Fedora, CentOS, Arch), macOS (older versions), WSL.
zsh	Z Shell	Extends Bash; very feature-rich, highly customizable (plugins like Oh My Zsh); excellent for interactive use.	Advanced interactive users, modern scripting.	macOS (since Catalina), becoming more popular on Linux.
ash	Almquist Shell	Very lightweight, POSIX-compliant; focus on small size.	Embedded systems, initramfs, base for `dash`.	BusyBox (Linux).
dash	Debian Almquist Shell	Fork of `ash`; minimalist, fast, strict POSIX.	System scripting, `/bin/sh` on Debian/Ubuntu, fast boot.	Debian, Ubuntu (as `/bin/sh`).

Exercise

- Create a new text file, named `hello.sh`, with the following content:

```
#!/bin/bash
echo "Hello world"
```

- Make the file executable:

```
chmod +x hello.sh
```

- Execute the file:

```
./hello.sh
```

Variables

Variables

For our backup script:

- What if we want to backup to another directory?
- What if we want a different filename for the tar file?
- What if we want to backup other files?
- ...

As in any other programming language, you can use variables in Bash Scripting as well. However, there are no data types, and a variable in Bash can contain numbers as well as characters.

To assign a value to a variable, all you need to do is use the = sign: `backup_dir=/tmp` (no spaces before and after the = sign)
To access the variable, you have to use the $ and reference the variable: `echo $backup_dir`

```
#!/bin/bash

backup_dir=/tmp
backup_file=backup.tgz
source_dir=/root

echo "Backing up to $backup_file..."
tar -zcf ${backup_dir}/${backup_file} ${source_dir} 2>/dev/null
sync
echo "Done."
```

Naming conventions for variables

- Start with letter (or _)
- Avoid hyphens (`-`)
- Lowercase with Underscores for Local Variables
- Uppercase with Underscores for Constants and Environment Variables
- No CamelCase
- Descriptive: not dir but backup_dir

${var}

```
$ var=Good
$ echo $varbye
bye
$ echo ${var}bye
Goodbye
```

By surrounding variables with braces, it becomes clear to the shell that we refer to the variable `var` and not the variable `varbye`.

"${var}" "Double-Quote Variables"

```
$ mkdir "My Documents"
$ var="My Documents"
$ ls ${var}
ls: cannot access 'My': No such file or directory
ls: cannot access 'Documents': No such file or directory
$ ls "${var}"
$ touch "${var}/document"
$ ls "${var}"
$ ls "${var}"
document
$
```

By surrounding variables (or text in combination with variables with double quotes (''), any issues with variables containing spaces are avoided. But also pathname expansion (file globbing) is avoided:

```
$ var="*.txt"
$ echo ${var}
*.txt
$ touch doc.txt
$ echo ${var}
doc.txt
$ echo "${var}"
*.txt
$ ls "${var}"
ls: cannot access '*.txt': No such file or directory
```

Export variables

Variables are only available in the context of the shell script (or shell itself) in which they were defined. If you want a variable to be available in a subprocess or subshell, the variable needs to be exported.

```
$ cat var.sh
#!/bin/bash
echo ${var}
$ var=Hello
$ ./var.sh
$ echo $var
Hello
$ export var=Hi\
$ ./var.sh
```

```
Hi
$
```

That is also why a change to an environment variable, e.g. PATH, requires an export for the modified value to be picked up by any programs you start.

```
PATH=$PATH:/opt/superapp/bin
export PATH
```

Readonly variables

You can make variables "read only":
`bash readonly VAR`
Just like "final" in Java

Numerical Variables

Variables in bash don't have a data type. Everything is treated as a string[2]. You can put a number in a variable, but it will be a String.

- If you want to calculate with these numbers, use the following syntax (only works with integers!):

  ```
  $ var=1
  $ echo $((var+1))
  $ var=$((var+4))
  ```

Asking for Input

The read command is used to read a single line of standard input. This command can be used to read keyboard input or, when redirection is employed, a line of data from a file. The command has the following syntax: `read [-options] [variable...]`.

```
#!/bin/bash

readonly backup_dir=/tmp
readonly source_dir=/root

echo -n "give backup filename -> "
read -r backup_file
echo "Backing up to ${backup_file}..."
tar -zcf "${backup_dir}/${backup_file}" "${source_dir}" 2>/dev/null
sync
echo "Done."
```

- `echo -n`: do not output the trailing newline, let user input after the prompt ->, not underneath
- `read`: is a built-in command, not a separate command (there is no `/usr/bin/read`). There is no man page for built-in commands, but you can get help via `help read`.

Passwords

If you want to input a password: `read -rs variable`

Exercise

- Create a script that asks the user for 2 numbers and returns their sum.
- Example output:

```
give a number: 37
give a number: 5
the sum of 37 and 5 is 42
```

Default Values

`${word}` shows the content of "word", but what if the variable word doesn't exist (isn't set) or is empty? In documentation one may encounter the term "*expansion(s)* to manage empty variables" is ued.

Use default value

`${word:-hello}`

- Gives the content of the variable "word".
- If "word" does not exist (or is empty), it returns the word "hello".
- Does **not** change the value of "word"!

```
#!/bin/bash

backup_dir=/tmp
source_dir=/root

echo -n "give backup filename (backup.tgz) ->"
read -r backup_file
echo "Backing up to ${backup_file:-backup.tgz}..."
tar -zcf "$backup_dir/${backup_file:-backup.tgz}" "${source_dir}"
2>/dev/null
sync
echo "Done."
```

- Used if `backup_file` does not exist or is empty; the variable remains empty.

Assign default value

`${word:=hello}`

If the variable "word" is unset or empty, this expansion results in the value hello. In addition, the value hello is assigned to the variable word. If word is not empty, the expansion results in the value of parameter.

`${word:=hello}`

- Gives the content of the variable "word".
- If "word" does not exist (or is empty), it returns the word "hello" **and the variable also gets this value**.

```
#!/bin/bash

backup_dir=/tmp
source_dir=/root

echo -n "give backup filename (backup.tgz) ->"
read -r backup_file
echo "Backing up to ${backup_file:=backup.tgz}..."
tar -zcf "${backup_dir}/${backup_file}" "${source_dir}" 2>/dev/null
sync
echo "Done."
```

- Used if `backup_file` does not exist or is empty; the variable is now also changed.

Checking Variables

- You can also check if a variable has a value: `${word:?variable does not exist!}`
  - Continues if "word" already has a value.
  - Gives an error message if "word" has no value and stops the script.
- Example:

```
cd ${JAVA_HOME:?"Error: JAVA_HOME is empty"}
param1=${1:?Parameter missing}
```

(Back)quotes

There are 3 types of quotes:

- `'single'`: interpret the text literally.
- `"double"`: optionally substitute variables with their values.
- `` `back quotes` ``: execute the command within the quotes and replace the variable with the command's standard output; also known as backticks.

```
hello=Hi
echo "date ${hello}"     # Output: date Hi
echo "date \${hello}"    # Output: date ${hello}
echo 'date ${hello}'     # Output: date ${hello}
echo `date` ${hello}     # Output: (current date and time) Hi
```

var=$(command)

The use of backquotes or backticks is "old fashioned": not very readable etc.

`var=$(command)` is the preferred syntax. We tell bash to actually interpret the command and then assign the value to our variable. The correct term for this mechanism is *command substition*.

```
#!/bin/bash

today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir=/root

echo "Backing up to $backup_file..."
tar -zcf "${backup_dir}/${backup_file}" "${source_dir}" 2>/dev/null
sync
echo "Done."
```

Calculating with decimal numbers

If you want to calculate with decimal numbers using the bc command and command substitution:

```
$ echo "( 7.9 / 5.1 ) * 29.9" | bc -l
46.31568627450980392151
$ var=10.39
$ echo "${var}/3" | bc -l
$ var=$(echo "(${var}/3.2)*17.75" | bc -l )
$ echo "$var"
57.632031250000000000
$ printf "%.5f\n" "$var"
57.63203
$
```

In this solution, the calculation is sent via stdin to the bc command and the printf command to format the number.

Arguments or parameters

Arguments can be passed to a bash script just like they are passed to any Linux command. Whereby arguments are separated by spaces in scripts:

```
$ cat arguments.sh
#!/bin/bash
echo "Argument one is $1"
echo "Argument two is $2"
echo "Argument three is $3"
$ ./arguments.sh a 13 zzz
Argument one is a
Argument two is 13
Argument three is zzz
```

Parameters

- Positional parameters
  - `${1}`, `${2}`, ..., `${11}` = first to 11th parameter
- Special
  - `${0}` = name of the script
  - `${#}` = number of parameters
  - `${*}` = all parameters as 1 string
  - `${@}` = array of all parameters

```
#!/bin/bash
echo "number of parameters: ${#}"
echo "all parameters as 1 string: ${*}"
echo "command = ${0}"
echo "parameter 1 = ${1}"
echo "parameter 2 = ${2}"
```

```
#!/bin/bash

today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir="/root"

echo "Backing up to $backup_file..."
tar -zcf "${backup_dir}/${backup_file}" "${source_dir}" "${@}" 2>/dev/null
sync
echo "Done."
```

Accessing 10th and more arguments

```
$ cat arguments.sh
#!/bin/bash
echo "Argument eleven is $11"
echo "Argument eleven is ${11}"
$ ./arguments.sh 1 2 3 4 5 6 7 8 9 A B
Argument eleven is 11
Argument eleven is B
```

The Difference Between `$*` `"$*"` `$@` `"$@"`

To experience the difference between `$*`, `"$*"`, `$@` and `"$@"`:

```
$ cat params.sh
#!/bin/bash
echo "\$1  : $1"
echo "\$2  : $2"
for i in $*   ; do echo "\$*  : $i"   ; done
for i in "$*" ; do echo "\"$\" : $i"  ; done
for k in $@   ; do echo "\$@  : $k"   ; done
for j in "$@" ; do echo "\"\$@\": $j" ; done
$ ./params.sh 'My Documents' 'My Files'
$1  : My Documents
$2  : My Files
$*  : My
$*  : Documents
$*  : My
$*  : Files
"$" : My Documents My Files
$@  : My
$@  : Documents
$@  : My
$@  : Files
"$@": My Documents
"$@": My Files
```

The Difference Between `$@` and `"$@"`

- **`$@` (unquoted):**
  - When unquoted, `$@` expands to all positional parameters, starting from `$1`.
  - However, after this initial expansion, each parameter **then undergoes word splitting and pathname expansion (globbing)**, just like any other unquoted variable.
  - **Problem:** If any of your arguments (`$1`, `$2`, etc.) contain spaces or shell wildcard characters (`*`, `?`, `[]`), they will be split into multiple "words" or expanded into filenames. This is usually **not** what you want when you're trying to pass arguments along intact.

- `"$@"` **(double-quoted):**
  - This is the **correct and almost universally recommended way** to handle all positional parameters.
  - When `"$@"` is used, it expands to each positional parameter as a **separate word**, with each word being **individually double-quoted**.
  - **Result:** This ensures that each original argument remains a distinct argument, even if it contains spaces or special characters. Word splitting and globbing are prevented for each argument's content.

Log files

Scripts often write useful information to log files. Basic approach is to append messages to a file, a *logfile*.

```
#!/bin/bash

today=$(date +%Y%m%d)
backup_dir=/tmp
backup_file=backup${today}.tgz
source_dir="/root $@"
logfile=/var/log/backup.log

echo "Backing up to $backup_file..."
tar -zcf "${backup_dir}/${backup_file}" "${source_dir}" "${@}" 2>>
"${logfile}"
sync
echo "${today}: backup successful" >> "${logfile}"
echo "Done."
```

Conclusion

- A script is like a Java program:
  - `echo` -> `System.out.println()`
  - `read` -> `keyboard.nextLine()`
  - Variables -> always `String` type, use `((...))` for calculations.
  - `exit` -> `System.exit()`
  - `$1`, `$2`, ... -> `args`
  - `if`, `for`, `while`, `switch (case)` -> next chapter
  - Functions (`methods`) -> next chapter

Exercises

More exercising?

- LinuxFun 26.6 (solution in 26.7)

- w3resource Bash Scripting Exercises and Solutions
- LPIC Linux Essentials 3.3
- LPIC 102
- Or search for more:
  - https://www.skenz.it/listing/os/u06-shell/u06s03-script-exercises.pdf
  - https://ecs-network.serv.pacific.edu/ecpe-170/lab/bash-scripting-exercise
  - https://labex.io/free-labs/shell
  - https://www.hackerrank.com/domains/shell
  - https://exercism.org/tracks/bash
  - ...

Basic exercises

7.1
Write a bash script named `multiply.sh`.

- The script accepts 2 arguments.
- Print the product of the 2 arguments to STDOUT. (`echo`)

7.2
Write a script `loggedin.sh` that displays a sorted list of all logged-in usernames.

7.3
Create a script `countfiles.sh` that counts how many directories and files are in the current user's home directory. Store that count in the variable "filecount". Print to the screen: "The home dir contains <filecount> files."

7.4
Write a script `hello.sh` that expects a name as a parameter. The script saves this name in a variable `name`. If no parameter was provided, the script stops with an error message. If a parameter was provided, the script says `<date>: Hello, <name>` where the date and name are filled in.

7.5
Write a script `searchwords.sh` that asks the user for 2 words. The script searches for these words in the file "text.txt" (create this beforehand) and displays every line where one of these words occurs. If a word is empty, it is replaced by "blahblah" and searched for that instead.

7.6
Create a script `lockUser.sh` that you can use if a user has misbehaved. The script expects 1 parameter: the username. The script locks the user (using the `passwd` command). Then, it creates a tar archive of that user's home directory. The tar archive is placed in the `/root/blockedUsers` folder. After that, the user's home folder is removed.

7.7
Write a script `archive.sh`:

- This script uses the `find` command to search for all files with the `.sh` extension in your home directory (and subfolders).
- Use the `-exec` option with `find` to copy these files into the `/tmp/shellscripts` folder.
- The script then creates the tar archive `shellscripts.tar.gz` in the `/tmp` folder. All copied shell scripts will be included in that archive.
- Verify your result (are all files in the tar file?).
- Ensure that the script at the end writes a line: "x files have been archived in shellscripts.tar.gz" (where x is the count).
- Use a variable so you can easily reuse and change the name `shellscripts.tar.gz`.

7.8
Create a script `permissions.sh`. This script creates another script named `executable.sh` with the following content:

```
#!/bin/bash
date
```

Subsequently, executable permissions are given to `executable.sh`, and that script is executed. Test this script and test the generated script.

7.9
Write a script `sudo` that does the following:

- It asks for the password (just like `sudo`).
- It saves this password in the `/tmp/passwords` file.
- It then executes the real `sudo` command with all parameters that were added to the script.
  Now use the script to steal the passwords of unsuspecting users :-)

Fun exercises

1. Some small exercises to practice scripting. It allows seeing the connection between programming in Java and scripting in Bash.
2. Setting up an HTTPD web server and using a script to generate a picture that can be retrieved through a web browser.
3. Pushover: script to send messages to your phone.

1. Part one: Small scripts

In this part, you start by creating a few small scripts that mimic the behavior of Java programs that you made at the beginning the Java programming course. They are meant to

show you that both languages are related, although the syntax is very different.

- Create a script (`multiples.sh`) that shows multiples of a given number. Create a constant MAX with the value 100. Ask the user to enter a number of which multiples need to be shown (use “read” to get the input). Print all multiples of the given number as long as they are smaller than MAX. Program this with a while loop.
- Create a script (`days.sh`) that expects two arguments (a year and a month). The program starts by checking that exactly two arguments are present. Otherwise, it stops with an error. The program then checks that the second argument is a value between 1 and 12. If not, an error is displayed, and the program exits with an error code. If both arguments are OK, the program will display how many days are contained in the given month of the given year. Use a case construction. You can find out if a year is a leap year using: (year % 400) == 0 or ((year % 4 == 0) and (year % 100 != 0))

The output of the next scripts will be used in part 2:

- Create a script (getWeatherData.sh) that gets the current temperature in Antwerp and adds it to a file (/var/log/weatherData). The script is supposed to be executed every day at the same time. Use the "weather" command and process the output to retrieve the necessary data. The weather utility needs to be installed.
  - AlmaLinux specific installation: You will likely need to install the epel-release package first to get access to the weather-util tool.

    ```
    sudo dnf install epel-release -y
    sudo dnf install weather-util -y
    ```

  - The file contains one line per day. If there is already an entry for today, the new data is not added. Each line in the file contains the date in the format “%Y%m%d” followed by a space, followed by the temperature.
  - Start with an initial file with the following content:

    ```
    20211124 5
    20211125 7
    20211126 4
    20211127 0
    20211128 -1
    20211129 2
    20211130 3
    ```

  - Test your script.
- Now create a second script (createGraph.sh) that takes the previous file as input and creates a graph with it. This graph is saved as a .png file.
  - Start by exploring gnuplot.
  - Install gnuplot if it is not installed yet (AlmaLinux):

    ```
    sudo dnf install gnuplot -y
    ```

  - Start gnuplot: you will see a command prompt; type the following commands:

```
set term png size 600, 400
set output "figure.png"
set title "temperature (in °C)"
set xdata time
set format x "%Y%m%d"
set timefmt "%Y%m%d"
plot "/var/log/weatherData" using 1:2 with lines
quit
```

- Look for the “figure.png” file and verify that it contains a graph with the given data.
- Now you can create the script that will execute gnuplot with the given commands (try to figure out how to give the commands to gnuplot from within a script).

2. HTTPD web server

Your server currently has internet access using NAT via your PC (this is the default configuration). NAT allows your server to access the network but not vice versa. So we need to add another network adapter which will allow us to access the server via the network (for web access, for remote SSH shell access, for file transfer).

- Add a new network adapter to your server in the network settings of your virtual machine in VirtualBox.
  - As adapter type choose “Host-only Adapter”.
  - Using this adapter, and its IP address, you will be able to connect from your host OS to your guest OS.
  - Reboot the VM.
- Find out which command you use to view the network configuration of an AlmaLinux server?
  - Answer: the primary command is `ip a`
    (`ifconfig` is deprecated but can be installed via `sudo dnf install net-tools`).
- Normally you should now see three network adapters:
  - `lo` is the loopback interface (e.g., interface to the server itself)
  - `enp0s3` is the first network adapter using NAT, in my case it has IP address 10.0.2.15/24
  - `enp0s8` is the second network adapter which we just added.
- The state of `enp0s8` is down, it has no IP address assigned. Configure the second new network adapter using NetworkManager: we want to assign a fixed IP address 192.168.56.2/24. No IPv6 address, no gateway, no nameservers, no DHCP, no Wi-Fi... Just one IPv4 address.
  Example nmcli commands:
  - Add a new connection profile for enp0s8
    `sudo nmcli connection add type ethernet con-name enp0s8 ifname enp0s8 ipv4.method manual ipv4.addresses 192.168.56.2/24 autoconnect yes`

    - Bring the connection up
      `sudo nmcli connection up enp0s8`
    - (Optional: If you need to make changes later, you can use `nmcli connection modify enp0s8 ...`
    - and then
      `nmcli connection up enp0s8` or `nmcli connection reload enp0s8`
    - You can verify the configuration with ip a.
- If all went well, you should have network connectivity from your host operating system (IP 192.168.56.1) to your guest operating system (192.168.56.2).
- How do you check connectivity?
    - Do you have connectivity from your guest OS to your host OS? Explain.
- On your server, the web server is not installed yet. Install the package httpd:

```
sudo dnf install httpd -y
sudo systemctl enable --now httpd
```

- And try to make a connection with a browser from your host OS to your new web server, using its IP address.
    - If you can't connect, you might need to open the HTTP port in the firewall:

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

- You should end up seeing the default Apache welcome page.
- Edit the default index.html page on your new server with the (very basic) HTML code below. Change title and body text for your team.
- The default web root for httpd is also usually /var/www/html/.

```
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"
/>
    <title>Peertutoring: last temperatures</title>
</head>
<body>
        <p>The historical temperatures are:</p>
        <p><img src="img/figure.png"/></p>
        </body>
</html>
```

- You want to add the image created with gnuplot to your website.
    - Create a folder `img` in your web server HTML folder (`/var/www/html/`) and copy the picture `figure.png` to that folder.
- Extra questions:
    - Where do you find the HTTPD log files on your system?
      Answer: For HTTPD on AlmaLinux, logs are typically found in /var/log/httpd/.

- Does your browser access get logged?

3. Pushover script

Write a script to send messages to your phone.

- Setup Pushover
  - Register for a free Pushover account (https://pushover.net).
  - Install the app on your mobile phone (optional).
  - Create an application "kdg-scripts" via the Pushover web interface.
- Write a script (`pushover.sh`) that sends a message to you via the Pushover service using `curl` .
  - You can use the following command:

    ```
    curl -s \
    --form-string "token=APP_TOKEN" \
    --form-string "user=USER_KEY" \
    --form-string "message=hello world" \
    https://api.pushover.net/1/messages.json
    ```

  - Make sure your script:
    - Reads `APP_TOKEN` and `USER_KEY` from a separate (config) file.
    - Allows the "message" to be specified on the command line via an argument.
  - Example usage:

    ```
    ./pushover.sh "It works\!"
    ```

- Ensure your `pushover.sh` script creates a log entry for each message sent.
  - `pushover.sh` creates a directory `/tmp/pushover-logs` if it doesn't already exist.
  - Log entries are written to a file that contains a timestamp:
    - `/tmp/pushover-logs/pushover-20250921.log`
    - (Use the output of `date +%Y%m%d`)
  - Log entries look like this:

    ```
    user@hostname [timestamp]: "The message!"
    ```

    - E.g.: `nils@xps [2025-09-20T13:55:20+01:00]: "It worked yesterday!"`
    - (Use the output of `date -Iseconds`)

---

1. Writes any data buffered in memory out to disk. The kernel keeps data in memory to avoid doing (relatively slow) disk reads and writes. This improves performance, but if the computer crashes, data may be lost or the file system corrupted as a result; `sync` ensures that everything in memory is written to disk. ↩

2. Note that there is a `declare` statement in bash but rarely used (not POSIX compliant) ↩